3.0 Condicionales e iterador FOR

IF, ELSE, ELIF, AND, OR y FOR

Programación I
Ingeniería Electrónica y Telecomunicaciones

Comisión 3
Profesor: Lautaro Linquimán
Universidad Nacional de Rio Negro

Repaso express de la clase anterior

- ¿Que es el intérprete interactivo de python?
- ¿Que función usamos para imprimir cosas en pantalla?
- ¿Que es una variable?
  - ¿Le podemos poner cualquier nombre?
- ¿Cuales son los tipos de variables?

ALGORITMOS

Aprendiendo a pensar como un programador

PENSAR ANTES DE PROGRAMAR

El trabajo de un programador no es solo escribir código, en la mayoría de los casos también se requiere o es conveniente realizar un diseño para obtener buenos resultados a la hora de programar.

El proceso formal del desarrollo de software tiene resumidamente las siguientes etapas:

Problema → Diseño → Implementación → Prueba

- **El problema** es lo que queremos resolver con el programa.
- **El diseño** nos permite descubrir cómo lo vamos a resolver, definiendo los pasos y la lógica antes de escribir código.
- **La implementación** es llevar ese diseño a un lenguaje de programación, escribiendo el código que ejecuta la solución.
- **La prueba** consiste en verificar que el programa funciona correctamente y resuelve el problema planteado.

Postdata: El proceso no suele ser tan lineal ni literal

Dependiendo del área y expertis se puede ver más así:
Problema → Diseño → Implementación → Prueba → Problemas de implementación → Google → Rediseño → Implementación → Más Pruebas → ~~Frustración~~ 😭 → Gratificación de un programa bien hecho (si anda ya no lo tocamos 🙄)

Para darle formalismo y seriedad lo llamamos **proceso iterativo incremental**.

ALGORITMOS

El algoritmo es una serie de pasos ordenados que nos permiten resolver un problema.

Los humanos ejecutamos algoritmos todo el tiempo, y somos bastante buenos para hacerlo.

Aunque no lo pensemos, tomamos decisiones constantemente en base a condiciones y eventos externos.

Imaginemos los pasos que hacemos para **cargar el celular**

1. Miramos el celular (**Acción**)
2. ¿La batería es menos a 20%? (**Condición**)
    a. **SI:** Cargamos
    b. **NO:** Seguimos mirando tik toks

DISEÑADORES DE ALGORITMOS

Nos ponemos el sombrero de un arquitecto de software que tiene que implementar el sistema de control de accesos de LELAC

Diseñar un algoritmo que determine si una persona puede entrar.

Datos a solicitar: edad y documento.

Aplicando lógica

En un lenguaje de programación

Operaciones lógicas

Aunque los programas parecen muy complejos —redes sociales, videojuegos, sistemas bancarios— en el fondo hay un concepto muy simple.

Las computadoras toman decisiones evaluando condiciones que solo puede tener dos resultados posibles:

- Verdadero (true)
- Falso (false)

Aunque se ven simples, el uso de estos dos estados permite construir cosas mucho más complejas:

- ¿La contraseña es correcta?
- ¿El usuario tiene permiso?
- ¿Hay stock disponible?
- ¿El personaje perdió toda su vida?
- ¿El sensor detectó movimiento?

Operadores de comparación

Los operadores de comparación nos permiten transformar números o textos en expresiones que pueden evaluarse como verdadero o falso.

¿un texto es igual a otro?

“hola mundo” == “hola mundo”

“hola mundo” == “chau mundo”

¿un número es igual a otro?

1 == 1

1 == 2

¿un número es mayor que otro?

1 > 2

3 > 2

> **Puedes copiar el texto en negrita en el intérprete y ver el resultado**

Tabla de operadores

Operación	Significado
<	estrictamente menor que
<=	menor o igual que
>	estrictamente mayor que
>=	mayor o igual que
==	igual que
! =	diferente que

Bloque condicional IF

Los resultados de las comparaciones de forma aislada no son muy útiles, pero combinado con funciones como if, es donde nace todo su potencial.

La traducción de “if” es “si” en español, imaginemos el acceso al banco verificando nuestra contraseña.

Si **Contraseña_Registrada** es igual a **Contraseña_ingresada** entonces permitir el acceso.

En el codigo se puede ver de la siguiente manera:

```
if contraseña_registrada == contraseña_ingresada:
    print(“Acceso permitido”)
```

Siempre que el valor booleano en el if sea verdadero, entonces su bloque contenido será ejecutado.

Todos los lenguajes de programación implementan de alguna manera la función if dado que es elemental para la creación de los programas.

Bloque condicional IF

Analicemos más en detalle el bloque condicional if.

```
contrasena_ingresada = "12345"
contrasena_almacenada = "12345"
acceso_permitido = False

if contrasena_almacenada == contrasena_ingresada:
    acceso_permitido = True
    print("Acceso permitido")

print("Gracias por usar el sistema seguro del banco")
```

Condición lógica

Bloque condicionado

Tabulación

Codigo no condicionado

Bloque IF – ¿Que hace este código?

```
if True:
    print("Codigo condicionado")

if False:
    print("Codigo condicionado")

print("Codigo sin condicion")
```

Extendiendo las condiciones: Bloques ELSE

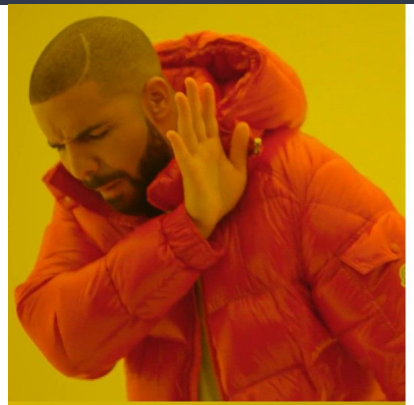
La condición else nos permite evaluar la condición que no se cumple dentro de un bloque condicional, es decir, lo opuesto a lo evaluado en el “if”, por ende “if” siempre debe existir previamente a “else”..

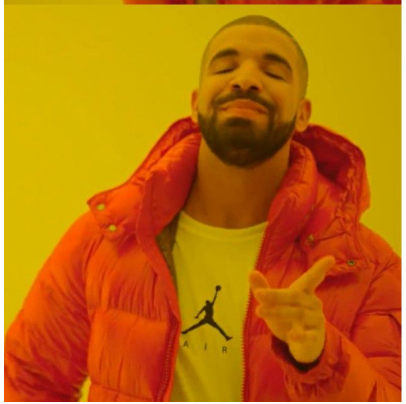
La traducción al español de “else” es “**de lo contrario**”

Extendamos el ejemplo del banco:

Si **Contraseña_Registrada** es igual a **Contraseña_ingresada** entonces **permitir el acceso**.
De lo contrario **denegar acceso**

Extendiendo las condiciones: Bloques ELSE





Si no tuvieramos else, deberiamos hacer operaciones logicas opuestas, lo cual es poco practico.

```
if contraseña_registrada == contraseña_ingresada:
        print("Acceso permitido")
if contraseña_registrada != contraseña_ingresada:
        print("Acceso denegado")
```

En el codigo lo podemos ver de la siguiente manera

```
if contraseña_registrada == contraseña_ingresada:
        print("Acceso permitido")
else:
        print("Acceso denegado")
```

Extendiendo las condiciones: Bloques ELIF

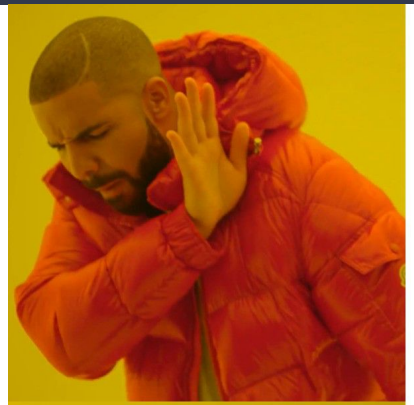
La condición **elif** es un abreviación del termino “else if”, nos permite evaluar una condición cuando otra no se cumple.

La traducción al español de “else if” es “**de lo contrario si**”

Extendamos el ejemplo del banco:

Si **Contraseña_Registrada** es igual a **Contraseña_ingresada** entonces **permitir el acceso**.
De lo contrario si **uso clave token** entonces **permitir el acceso**

Extendiendo las condiciones: Bloques ELIF



Si no tuvieramos elif, deberiamos hacer operaciones logicas anidadas con else,, lo cual es poco practico.

```
if contraseña_registrada == contraseña_ingresada:
        print("Acceso permitido")
else:
        if uso_clave_token == True:
                print("Acceso permitido")
```

Con elif queda de la siguiente manera

```
if contraseña_registrada == contraseña_ingresada:
        print("Acceso permitido")
elif uso_clave_token == True:
        print("Acceso permitido")
```

Bloque IF ELIF ELSE

```python
clave_almacenada = "1234"
clave_ingresada = "1234"
uso_clave_token = False

if clave_almacenada == clave_ingresada:
  print("Acceso con clave")
elif uso_clave_token:
  print("Acceso permitido")
else:
  print("Acceso denegado")
```

https://pastebin.com/UmNQRMvY

Operadores logicos

Operadores lógicos

En la práctica las operaciones lógicas no suelen ser una sola comparación, sino la combinación de varias condiciones. Para esto existen los **operadores lógicos**.

Los operadores lógicos permiten **combinar valores booleanos (verdadero o falso)** y obtener un nuevo resultado booleano.

Los dos operadores principales son **AND** (y) y **OR** (o).

Ejemplo con AND:
"¿Es mayor de 18?" **y** "¿Tiene licencia de conducir?"
Si **ambas condiciones son verdaderas**, entonces puede manejar.

Ejemplo con OR:
"¿Hace frío?" **o** "¿Está lloviendo?"
Si **al menos una de las dos condiciones es verdadera**, conviene llevar abrigo.

Operadores Lógicos

Ejemplo con AND:
"¿Es mayor de 18?" **y** "¿Tiene licencia de conducir?"
Si **ambas condiciones son verdaderas**, entonces puede manejar.

En el código se ve de la siguiente manera:

```python
edad = 19
es_mayor_de_18 = edad > 18
tiene_carnet = True
if(es_mayor_de_18 and tiene_carnet):
    print("Puede manejar")
```

```python
edad = 19
tiene_carnet = True
if(edad > 18 and tiene_carnet):
    print("Puede manejar")
```

```python
if(19 > 18 and True):
    print("Puede manejar")
```

Operadores lógicos

Ejemplo con OR:
"¿Hace frío?" **o** "¿Está lloviendo?"
Si **al menos una de las dos condiciones es verdadera**, conviene llevar abrigo.

```
hace_frio = True
esta_lloviendo = False
if(hace_frio or esta_lloviendo):
    print("Llevar abrigo")
```

```
hace_frio = False
esta_lloviendo = True
if(hace_frio or esta_lloviendo):
    print("Llevar abrigo")
```

```
if(True or False):
    print("Llevar abrigo")
```

Operadores logicos - ¿Que pasa aqui?

```
clave_almacenada = "1234"
clave_ingresada = "1234"
uso_clave_token = False

if clave_almacenada == clave_ingresada and uso_clave_token:
  print("Acceso permitido")
else:
  print("Acceso denegado")

print("Gracias por usar el sistema seguro del banco")
```

Operadores logicos - ¿Que pasa aqui?

```
monto = 900
es_cliente_vip = False
tiene_cupon = True

if monto > 1000 and es_cliente_vip or tiene_cupon:
    print("Se aplica descuento")
else:
    print("No hay descuento")
```

Operadores – Orden de precedencia

Al igual que en las operaciones matemáticas, las operaciones lógicas dentro de python también tiene precedencia.

1. El parentesis “( )”
2. Operaciones comparativas “== != > < >= <=”
3. Operador AND
4. Operador OR

Entonces, ¿que paso en el ejemplo anterior?

```
monto = 900
es_cliente_vip = False
tiene_cupon = True

if (monto > 1000 and es_cliente_vip) or tiene_cupon:
    print("Se aplica descuento")
else:
    print("No hay descuento")
```

Aparecieron unos parentesis magicamente

BUCLE FOR

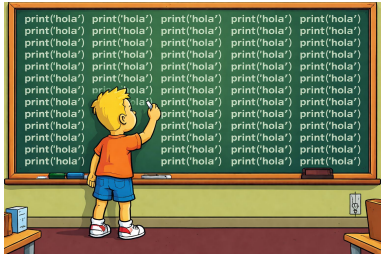
Bucle FOR

El bucle for nos permite realizar repeticiones de diferentes tipos.

Siendo la más simple una repetición numérica, es decir repetir algo N cantidad de veces.

Supongamos que queremos saludar a 100 personas, como ¿lo haríamos?

La primera idea que puede ocurrir es imprimir 100 veces hola.



Pero con el bucle for podes simplificar de la siguiente manera:

Repetir Cien veces
    Hola

En el código se ve de la siguiente manera:

```
for _ in range(0, 100):
    print("Hola")
```

Si quisiéramos ver cuantas personas saludamos podemos expresarlo de la siguiente manera:

```
for i in range(0, 100):
    print("Hola " + str(i + 1))
```

Bucle For – Código Anidado

El bucle for nos permite ejecutar código, es decir podemos poner condicionales y variables.

```
for alumno in range(0, 30):
    alumno_id = alumno + 1
    if (alumno % 2):
        print(f"Alumno {alumno_id} aprobado")
    else:
        print(f"Alumno {alumno_id} desaprobado")
```

Es importante la tabulación.

Ejemplos de uso:

range(5): Repite 5 veces empezando desde 0.
range(1, 5): Repite 4 veces empezando desde 1.
range(-10, 0): Nos permite recorrer 10 posiciones iniciando con un número negativo.
range(0, 10, 2): Nos permite hacer saltos de dos en dos, pueden cambiar el dos por otro número y ver que pasa.
range(0, 10, -1): Nos permite crear una cuenta regresiva.

Proba los ranges con el siguiente codigo y observa que pasa:

```
for i in range(.....):
        print(i)
```

Ejercicio

Juan, Pedro y Mateo tienen que lavar 6 platos.

Lavan un plato cada uno

Imprimir cada vez que uno de ellos lave un plato indicando su nombre.

Nuevo requerimiento: indicar cuantos platos lavo cada uno al finalizar.

Nuevo requerimiento: Ahora tienen que lavar 10 platos.

Nuevo requerimiento: Ahora tienen que lavar 100 platos.

TAREA

1.

Todo el material en https://github.com/Ymil/unrn-programacion-1/

FIN

Volvemos el miércoles a las 17